

AI51701/CSE71001 Assignment 2

word2vec

Due: Oct. 4, 2026, 11:59:00pm KST

Submission Instructions: You shall submit this assignment on BlackBoard as two submission files – your code as `assignment2coding.zip` and write up for `assignment2.pdf`; Run the `collect_submission.sh` script to produce your `assignment2coding.zip` file. It is your responsibility to make sure your code is runnable. If your code is not runnable, you will get no point.

Collaboration Policy: You are welcome to discuss assignments with others in the course, but solutions and code must be written individually.

1 Written: Understanding word2vec (60 points)

Recall that the key insight behind word2vec is that *a word is known by the company it keeps*. Concretely, consider a ‘center’ word c surrounded before and after by a context of a certain length. We term words in this contextual window ‘outside words’ (O). For example, in Figure 1, the context window length is 2, the center word c is ‘banking’, and the outside words are ‘turning’, ‘into’, ‘crises’, and ‘as’:

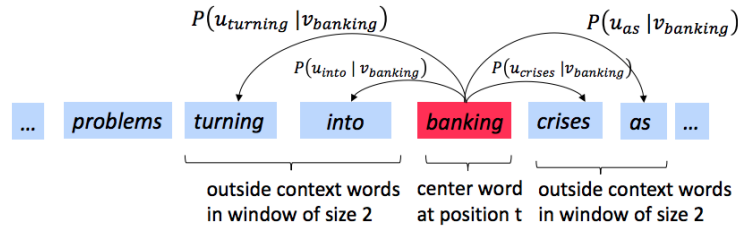


Figure 1: The word2vec skip-gram prediction model with window size 2

Skip-gram word2vec aims to learn the probability distribution $P(O|C)$. Specifically, given a specific word o and a specific word c , we want to predict $P(O = o|C = c)$: the probability that word o is an ‘outside’ word for c (i.e., that it falls within the contextual window of c). We model this

probability by taking the softmax function over a series of vector dot-products:

$$P(O = o|C = c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad (1)$$

For each word, we learn vectors u and v , where $\mathbf{u}_o$ is the ‘outside’ vector representing outside word o , and $\mathbf{v}_c$ is the ‘center’ vector representing center word c . We store these parameters in two matrices, $\mathbf{U}$ and $\mathbf{V}$. The columns of $\mathbf{U}$ are all the ‘outside’ vectors $\mathbf{u}_w$; the columns of $\mathbf{V}$ are all of the ‘center’ vectors $\mathbf{v}_w$. Both $\mathbf{U}$ and $\mathbf{V}$ contain a vector for every $w \in \text{Vocabulary}$ ¹.

Recall, for a single pair of words c and o , the loss is given by:

$$\mathbf{J}_{\text{naive-softmax}} = -\log P(O = o|C = c). \quad (2)$$

We can view this loss as the cross-entropy² between the true distribution $\mathbf{y}$ and the predicted distribution $\hat{\mathbf{y}}$, for a particular center word c and a particular outside word o . Here, both $\mathbf{y}$ and $\hat{\mathbf{y}}$ are vectors with length equal to the number of words in the vocabulary. Furthermore, the k^{th} entry in these vectors indicates the conditional probability of the k^{th} word being an ‘outside word’ for the given c . The true empirical distribution $\mathbf{y}$ is a one-hot vector with a 1 for the true outside word o , and 0 everywhere else, for this particular example of center word c and outside word o ³. The predicted distribution $\hat{\mathbf{y}}$ is the probability distribution $P(O|C = c)$ given by our model in equation 1.

(a) (6 points) Prove that the naive-softmax loss (Equation 2) is the same as the cross-entropy loss between $\mathbf{y}$ and $\hat{\mathbf{y}}$, i.e. (note that $\mathbf{y}$, $\hat{\mathbf{y}}$ are vectors and $\hat{\mathbf{y}}_o$ is a scalar):

$$-\sum_{w \in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w) = -\log(\hat{\mathbf{y}}_o). \quad (3)$$

Your answer should be one line. You may describe your answer in words.

(b) (10 points) Compute the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{v}_c$. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{U}$. Additionally, answer the following two questions with one sentence each: (1) When is the gradient zero? (2) Why does subtracting this gradient, in the general case when it is nonzero, make $\mathbf{v}_c$ a more desirable vector (namely, a vector closer to outside word vectors in its window)?

¹Assume that every word in our vocabulary is matched to an integer number k . Bolded lowercase letters represent vectors. $\mathbf{u}_k$ is both the k^{th} column of $\mathbf{U}$ and the ‘outside’ word vector for the word indexed by k . $\mathbf{v}_k$ is both the k^{th} column of $\mathbf{V}$ and the ‘center’ word vector for the word indexed by k . In order to simplify notation we shall interchangeably use k to refer to word k and the index of word k .

²The cross-entropy loss between the true (discrete) probability distribution p and another distribution q is $-\sum_i p_i \log(q_i)$.

³Note that the true conditional probability distribution of context words for the entire training dataset would not be one-hot.

- Note: Your final answers for the partial derivative should follow the shape convention: the partial derivative of any function $f(x)$ with respect to x should have the same shape as x .⁴
- Please provide your answers for the partial derivative in vectorized form. For example, when we ask you to write your answers in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{U}$, you may not refer to specific elements of these terms in your final answer (such as $\mathbf{y}_1, \mathbf{y}_2, \dots$).

(c) (10 points) Compute the partial derivatives of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to each of the ‘outside’ word vectors, $\mathbf{u}_w$ ’s. There will be two cases: when $w = o$, the true ‘outside’ word vector, and $w \neq o$, for all other words. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{v}_c$. In this subpart, you may use specific elements within these terms as well (such as $\mathbf{y}_1, \mathbf{y}_2, \dots$). Note that $\mathbf{u}_w$ is a vector while $\mathbf{y}_1, \mathbf{y}_2, \dots$ are scalars.

(d) (2 point) Write down the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{U}$. Please break down your answer in terms of $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_1}, \frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_2}, \dots, \frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_{|\text{Vocab}|}}$. The solution should be one or two lines long.

(e) (4 points) The ReLU (Rectified Linear Unit) activation function is given by Equation 4:

$$f(x) = \max(0, x) \quad (4)$$

Please compute the derivative of $f(x)$ with respect to x , where x is a scalar. You may ignore the case that the derivative is not defined at 0.⁵

(f) (6 points) The sigmoid function is given by Equation 5:

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \quad (5)$$

Please compute the derivative of $\sigma(x)$ with respect to x , where x is a scalar. Hint: you may want to write your answer in terms of $\sigma(x)$.

(g) (12 points) Now we shall consider the Negative Sampling loss, which is an alternative to the Naive Softmax loss. Assume that K negative samples (words) are drawn from the vocabulary. For simplicity of notation we shall refer to them as $w_1, w_2, \dots, w_K$, and their outside vectors as $\mathbf{u}_{w_1}, \mathbf{u}_{w_2}, \dots, \mathbf{u}_{w_K}$.⁶ For this question, assume that the K negative samples are distinct. In other words, $i \neq j$ implies $w_i \neq w_j$ for $i, j \in \{1, \dots, K\}$. Note that $o \notin \{w_1, \dots, w_K\}$. For a center word c and an outside word o , the negative sampling loss function is given by:

$$\mathbf{J}_{\text{neg-sample}}(\mathbf{v}_c, o, \mathbf{U}) = -\log(\sigma(\mathbf{u}_o^\top \mathbf{v}_c)) - \sum_{s=1}^K \log(\sigma(-\mathbf{u}_{w_s}^\top \mathbf{v}_c)) \quad (6)$$

⁴This allows us to efficiently minimize a function using gradient descent without worrying about reshaping or dimension mismatching. While following the shape convention, we’re guaranteed that $\theta := \theta - \alpha \frac{\partial \mathbf{J}(\theta)}{\partial \theta}$ is a well-defined update rule.

⁵If you’re interested in how to handle the derivative at this point, you can read more about the notion of subderivatives.

⁶Note: In the notation for parts (g) and (h), we are using words, not word indices, as subscripts for the outside word vectors.

for a sample $w_1, \dots, w_K$, where $\sigma(\cdot)$ is the sigmoid function.⁷

- (i) Please repeat parts (b) and (c), computing the partial derivatives of $\mathbf{J}_{\text{neg-sample}}$ with respect to $\mathbf{v}_c$, with respect to $\mathbf{u}_o$, and with respect to the s^{th} negative sample $\mathbf{u}_{w_s}$. Please write your answers in terms of the vectors $\mathbf{v}_c$, $\mathbf{u}_o$, and $\mathbf{u}_{w_s}$, where $s \in [1, K]$. Note: you should be able to use your solution to part (f) to help compute the necessary gradients here.
- (ii) In lecture, we learned that an efficient implementation of backpropagation leverages the re-use of previously-computed partial derivatives. Which quantity could you reuse between the three partial derivatives to minimize duplicate computation? Write your answer in terms of $\mathbf{U}_{o, \{w_1, \dots, w_K\}} = [\mathbf{u}_o, -\mathbf{u}_{w_1}, \dots, -\mathbf{u}_{w_K}]$, a matrix with the outside vectors stacked as columns, and $\mathbf{1}$, a $(K+1) \times 1$ vector of 1's.⁸
- (iii) Describe with one sentence why this loss function is much more efficient to compute than the naive-softmax loss.

Caveat: So far we have looked at re-using quantities and approximating softmax with sampling for faster gradient descent. Do note that some of these optimizations might not be necessary on modern GPUs and are, to some extent, artifacts of the limited compute resources available at the time when these algorithms were developed.

(h) (4 points) Now we will repeat the previous exercise, but without the assumption that the K sampled words are distinct. Assume that K negative samples (words) are drawn from the vocabulary. For simplicity of notation we shall refer to them as $w_1, w_2, \dots, w_K$ and their outside vectors as $\mathbf{u}_{w_1}, \dots, \mathbf{u}_{w_K}$. In this question, you may not assume that the words are distinct. In other words, $w_i = w_j$ may be true when $i \neq j$ is true. Note that $o \notin \{w_1, \dots, w_K\}$. For a center word c and an outside word o , the negative sampling loss function is given by:

$$\mathbf{J}_{\text{neg-sample}}(\mathbf{v}_c, o, \mathbf{U}) = -\log(\sigma(\mathbf{u}_o^T \mathbf{v}_c)) - \sum_{s=1}^K \log(\sigma(-\mathbf{u}_{w_s}^T \mathbf{v}_c)) \quad (7)$$

for a sample $w_1, \dots, w_K$, where $\sigma(\cdot)$ is the sigmoid function.

Compute the partial derivative of $\mathbf{J}_{\text{neg-sample}}$ with respect to a negative sample $\mathbf{u}_{w_s}$. Please write your answers in terms of the vectors $\mathbf{v}_c$ and $\mathbf{u}_{w_s}$, where $s \in [1, K]$. Hint: break up the sum in the loss function into two sums: a sum over all sampled words equal to w_s and a sum over all sampled words not equal to w_s . Notation-wise, you may write ‘equal’ and ‘not equal’ conditions below the summation symbols, such as in Equation 8.

(i) (6 points) Suppose the center word is $c = w_t$ and the context window is $[w_{t-m}, \dots, w_{t-1}, w_t, w_{t+1}, \dots, w_{t+m}]$, where m is the context window size. Recall that for the skip-gram version of word2vec, the total loss

⁷Note: The loss function here is the negative of what Mikolov et al. had in their original paper, because we are doing a minimization instead of maximization in our assignment code. Ultimately, this is the same objective function.

⁸Note: NumPy will automatically broadcast 1 to a vector of 1's if the computation requires it, so you generally don't have to construct 1 on your own during implementation.

for the context window is:

$$\mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U}) = \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U}) \quad (8)$$

Here, $\mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ represents an arbitrary loss term for the center word $c = w_t$ and outside word w_{t+j} . $\mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ could be $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ or $\mathbf{J}_{\text{neg-sample}}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$, depending on your implementation. Write down three partial derivatives:

- (i) $\frac{\partial \mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U})}{\partial \mathbf{U}}$
- (ii) $\frac{\partial \mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U})}{\partial \mathbf{v}_c}$
- (iii) $\frac{\partial \mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U})}{\partial \mathbf{v}_w}$ when $w \neq c$

Write your answers in terms of $\frac{\partial \mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})}{\partial \mathbf{U}}$ and $\frac{\partial \mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})}{\partial \mathbf{v}_c}$. This is very simple – each solution should be one line. Once you’re done: Given that you computed the derivatives of $\mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ with respect to all the model parameters $\mathbf{U}$ and $\mathbf{V}$ in parts (a) to (c), you have now computed the derivatives of the full loss function $\mathbf{J}_{\text{skip-gram}}$ with respect to all parameters. You’re ready to implement `word2vec`!

2 Coding: Implementing word2vec (40 points)

In this part you will implement the word2vec model and train your own word vectors with stochastic gradient descent (SGD). Before you begin, first run the following commands within the assignment directory in order to create the appropriate conda virtual environment. This guarantees that you have all the necessary packages to complete the assignment. Also note that you probably want to finish the previous math section before writing the code since you will be asked to implement the math functions in Python. You’ll probably want to implement and test each part of this section in order, since the questions are cumulative.

```
conda env create -f env.yml
conda activate a2
```

Once you are done with the assignment you can deactivate this environment by running:

```
conda deactivate
```

For each of the methods you need to implement, we included approximately how many lines of code our solution has in the code comments. These numbers are included to guide you. You don’t have to stick to them, you can write shorter or longer code as you wish. If you think your implementation is significantly longer than ours, it is a signal that there are some `numpy` methods you could utilize to make your code both shorter and faster. `for` loops in Python take a long time to complete when

used over large arrays, so we expect you to utilize `numpy` methods. We will be checking the efficiency of your code. Note: If you are using Windows and have trouble running the `.sh` scripts used in this part, we recommend trying `Gow` or manually running commands in the scripts.

- (a) (24 points) We will start by implementing methods in `word2vec.py`. You can test a particular method by running `python word2vec.py m` where `m` is the method you would like to test. For example, you can test the sigmoid method by running `python word2vec.py sigmoid`.
 - (i) Implement the `sigmoid` method, which takes in a vector and applies the sigmoid function to it.
 - (ii) Implement the softmax loss and gradient in the `naiveSoftmaxLossAndGradient` method.
 - (iii) Implement the negative sampling loss and gradient in the `negSamplingLossAndGradient` method.
 - (iv) Implement the skip-gram model in the `skipgram` method.

When you are done, test your entire implementation by running `python word2vec.py`.

- (b) (8 points) Complete the implementation for your SGD optimizer in the `sgd` method of `sgd.py`. Test your implementation by running `python sgd.py`.
- (c) (8 points) Now we are going to load some real data and train word vectors with everything you just implemented! We are going to use the Stanford Sentiment Treebank (SST) dataset to train word vectors. You will need to fetch the datasets first. To do this, run `sh get_datasets.sh`. There is no additional code to write for this part; just run `python run.py`. Note: The training process may take a long time depending on the efficiency of your implementation and the compute power of your machine (an efficient implementation takes one to two hours). Plan accordingly! After 40,000 iterations, the script will finish and a visualization for your word vectors will appear. It will also be saved as `wordvectors.png` in your project directory. Include the plot in your homework write up. In at most three sentences, briefly explain what you see in the plot. This may include, but is not limited to, observations on clusters and words that you expect to cluster but do not.